Java =>

=> Java -> Foundation

=> 1995 => 2023

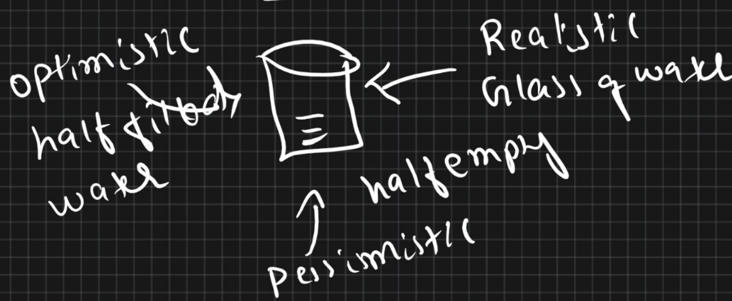
=> Object Orientation :-

Orientation:-

↳ prospective

↳ way of looking

↳



=> Object => Laptop, mic, speaker, Room, House, pen, Book, Bus, Road, Traine, Learner

=> Focus is the key:-

=>

=> whole world as collection object

=> Every object has something, Does something

⇓ properties

⇓ Behaival

=> Dog

has
- name
- age
- cost
- color

Does
- Bark()
- eat()
- sleep()

=> Object => class :-

=> Bike

cost　　　starts()　　　class Bike
color　　　stops()　　　{
gcal　　　ride()　　　　int cost;　　has
model　　　carryPeople()　　String color;
　　　　　　　　　　　　String model;
Datatypes　　method
variables　　　　　　　　void ride()　　does
　　　　　　　　　　　　{
　　　　　　　　　　　　}
　　　　　　　　　　　}

=> Variables and Datatypes　}

=> Info / Data => ✓　　age [18]

variable :-　　　　　　name [Rohan]

{ age = 18;
name = "Rohan";

=> statically typed PL (Strongly typed)　vs　Dynamically typed PL (weakly typed)

int age = 18; ✗　　　　✗ age = 18; ✓

char a = 'A';　　　　　✗ a = 'A'

String n = "Rohan";　　　JavaScript, Python

=> C, C++, Java ----

=> Working with Data / Info
↓
Type of Data => ✓
↓
Datatypes

=> To Manage Data { In-Built classes }
↓
Type of data { primitive Datatypes }

=> Character => 'a' '@', ' '

- char => Manages Character type Data
- byte, short, int, long => Manages Number (Integer)
- float, double => Real Number
  85.5
  92.92 } 85, 29
- boolean => true, false

char c = 'a';

int temp = 44;

double n = 44.5;

boolean a = false;

{ temp, count, flag }

Memory :- 0's and 1's 0101010

American CS

Character	Binary
A	0
B	1
A	00
B	01
C	10
D	11

A	000
B	001
C	010
D	100
E	101
F	110
G	
H	

128 => characters

$128 \Rightarrow 2^7 \Rightarrow 7$ bit

1 Byte => 8 bit

@
+
-
A
B

0	00000 000
1	0000 0001
2	
9	
A	
B	
Z	
a	
z	
@	1111 1111

ASCII } 128

A -> 8bit

B -> 8bit => 1 Byte

=> ASCII => Binary Representation to 128 characters

0 - 9
A - Z
a - z
@ # =

} 128   128   128

0,1 -> 8 bit => 1 Byte

C lang => ASCII => character

1 Byte => 8 bit

IEEE => Research => 65536 => characters

{ Unicode
{ UTF => Binary rep

128 + 65408 => 65536

0000 0000 0000 0000

8 bit -> 1 Byte
16 bit => 2 Bytes   4 2 Bytes

=> 128 -> ASCII + 65408

Char	Dec	Binary
A	65	16 bits
B	66	
C	67	
:	:	
a	97	
b		
c		

128

Char Dec
A = 65
: 66
Z
a
z

=> To manage character type Data Java supports 'char' primitive Data type
-> 2 Bytes

char c = 'a'; ✓   char a = 'ab'; ✗

=> Numbers, Integers : 45, 100, 4554L
byte   short   int   long

byte a = 45; short b = 45; int c = 45; long d = 45L;

1 Byte 1 Byte => 8 bits   2 Byte   16 bits   4 Byte int   8 Byte = long

-2^{n-1} to $+2^{n-1}-1$

-2^{8-1} to $+2^{8-1}-1$

-128 to 127

-2^{15} to $2^{15}-1$

32768 to 32767

2 Byte

byte a = 45;

1 Byte   3 Bytes

int n = 45;

4 byte

int

1.1
1.2
1.3
1.4
5
6
7
8

byte a = 45;
byte b = 45;
a+b;
45 + 45;
int <- 90

=> Datatypes =>

int :-

45.5 => float -> 4 bytes
double -> 8 Bytes

{ range of float & double

double a = 45.5;
float b = 45.5f;

45.54231

7 precision

14 digit precision

8 Bytes
a | 45.5 |
b | 45.5 |
4 Bytes

boolean a = true;
false;

=> char => 2 bytes
byte -> 1 byte
short -> 2 bytes
int -> 4 bytes
long -> 8 bytes

float -> 4 bytes
double -> 8 bytes
boolean => true / false

"Rohan" => String

=> instance variable:-

int a; 0 -
0.0 -
false -

boolean a; a [false]

double a; a [0.0]

=>
```
class Telusko  -> Identifiers
{
  int age;
  float avg;
  void conduct(bless l)
  {
  }
}
```

names -> logical
meaningful
Except keyword / Reserved words

int
float
double
class
short

try
catch
throw
throws

final
abstract
finally
for
while
static

=>
```
class int X
{

   int class = 10; X

}
```

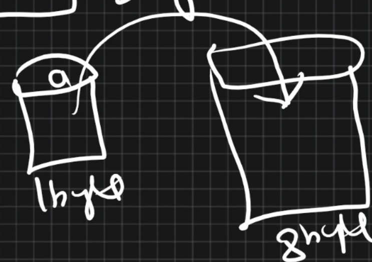
=> byte -> short -> int -> long -> float -> double
1B 2B 4By 8 4 8

int -> char 2By

byte a = 45;   a $\boxed{45}$ 1Byt

double b;   b $\boxed{}$ 8 byts

b = a;



S.o.p(a);
S.o.p(b);

double a = 45.5;
byte b;
b = a;

variable names

@ # * / _ $! % &

_ (underscore) | $ -> variable creation

int a = 5;   a $\boxed{\cancel{5}6}$ int a = 5;

a = a + 1;   =   a - 1;

a++; ->

int a = 5;   a $\boxed{\cancel{5}4}$

a = a - 1;

4

a--; ->

Post increment   Pre incre

var++   ++var

Post Dec   Pre Dece

var--   --var

++ -> Increment
-- -> Decrem

var++ -> post
--var

int a = 5; a $\boxed{\cancel{5}\ 6}$

int b; b $\boxed{5}$

b = a++;

s.o.p(a); 6

s.o.p(b); 5

int a = 5; a $\boxed{\cancel{5}\ 6}$

int b; b $\boxed{6}$

b = ++a;

s.o.p(a);

s.o.p(b);

int a = 5; a $\boxed{\cancel{5}\ \cancel{6}\ \cancel{5}\ \cancel{4}\ \cancel{7}\ 6}$

int b; b $\boxed{}$

$$b = \underset{6}{++a} + \underset{5}{--a} + \underset{5}{a--} + \underset{5}{++a} + \underset{5}{a++};$$

s.o.p(a); ~~8~~ 6

b 26

(-)

a < b   a++ } unary operator

int a = 5

$$\text{int } b = \underset{5}{a++} + \underset{7}{++a} + \underset{7}{a--} + \underset{6}{a--} - \underset{5}{a++} - \underset{7}{++a};$$

$\boxed{\cancel{5}\ \cancel{7}\ \cancel{6}\ \cancel{5}}$ 6

~~7~~ b

13

=> Arithmatic operators

+ - * / %

10+10=20

20-10=10

10*5=50

10/2=5

10%2=0

2)10(5
  10

  0

10%3

3)10(3
  9

  1

=> Relational operators :-

10 >
10 <

> < == <= >= => result in boolean value

10 > 10 } false

10 < 10 } f

10 == 10 } true

10 <= 10 ; T

10 >= 10 ; T

=> logical operators

AND | &&
OR | ||
NOT | !

OR

cond1	cond2	cond3	cond4	r
T	T	T	T	T
T	T	F	F	T
F	F	F	T	T
F	F	F	F	F

F F T
10 > 10 || 10 < 10 || 10 == 10 => true

10 >= 10 || 10 <= 10 || 10 == 10 => T
T T T

10 >= 10 || 10 < 10 || 10 == 10 } => T
T F T

10 < 10 || 10 > 10 || 10 > 12

F f f => falls

OR => || = XOR

AND :- &&

cond1	cond^n2	cond3	resul
T	T	T	T
T	T	F	f
F	T	T	F
F	f	f	F

10 > 10 && 10 < 10 && 10 == 10

F f T => f

10 >= 10 && 10 < 10 && 10 == 10 =>

T f T => false

10 >= 10 && 10 <= 10 && 10 == 10 =>

T && T && T => t =>

! NOT !T = f !f → T

!T → f !f => T

}=> **Conditionals**

=> if we want to perform activity / operation Based on condition

if-else => Ternary operator.

if - else => conditionals => T

```
if ()
{
}
else
{
}
```

single line :-

(cond^n) ? exp : exp

(with T above the first exp and F above the second)

(10>10) ? - : - ;

```
int a = 10;
int b = 20;
int c;
c = (a>b) ? a : b;
```